

Computer Architecture

CS 622 / EE 520

Fall 2026

Lecture 2
Shahid Masud

- Look at 3 Papers for upcoming Computer Architectures
- Evolution in Computing in post Moore's Law era
- RISC-V Instruction Set Architecture
- Introducing Computer Performance

Parallelism in Computer Architecture

Amdahl's Law (Section 1.9) prescribes practical limits to the number of useful cores per chip. If 10% of the task is serial, then the maximum performance benefit from parallelism is 10 no matter how many cores you put on the chip.

Some part of code can be processed in parallel
The remaining part of code must be processed in a sequential fashion

Amdahl's Law – Parallel or Accelerator



Amdahl's Law (Section 1.9) prescribes practical limits to the number of useful cores per chip. If 10% of the task is serial, then the maximum performance benefit from parallelism is 10 no matter how many cores you put on the chip.

Amdahl's Law defines the *speedup* that can be gained by using a particular feature. What is speedup? Suppose that we can make an enhancement to a computer that will improve performance when it is used. Speedup is the ratio

$$\text{Speedup} = \frac{\text{Performance for entire task using the enhancement when possible}}{\text{Performance for entire task without using the enhancement}}$$

Example of Performance Gains through Architecture

Table 1. Speedups from performance engineering a program that multiplies two 4096-by-4096 matrices. Each version represents a successive refinement of the original Python code. "Running time" is the running time of the version. "GFLOPS" is the billions of 64-bit floating-point operations per second that the version executes. "Absolute speedup" is time relative to Python, and "relative speedup," which we show with an additional digit of precision, is time relative to the preceding line. "Fraction of peak" is GFLOPS relative to the computer's peak 835 GFLOPS. See Methods for more details.

Version	Implementation	Running time (s)	GFLOPS	Absolute speedup	Relative speedup	Fraction of peak (%)
1	Python	25,552.48	0.005	1	—	0.00
2	Java	2,372.68	0.058	11	10.8	0.01
3	C	542.67	0.253	47	4.4	0.03
4	Parallel loops	69.80	1.969	366	7.8	0.24
5	Parallel divide and conquer	3.80	36.180	6,727	18.4	4.33
6	plus vectorization	1.10	124.914	23,224	3.5	14.96
7	plus AVX intrinsics	0.41	337.812	62,806	2.7	40.45

AVX = Intel Advanced Vector Extension

Types of Parallelism in Parallel Computers



Parallelism at multiple levels is now the driving force of computer design across all four classes of computers, with energy and cost being the primary constraints. There are basically two kinds of parallelism in applications:

1. *Data-level parallelism (DLP)* arises because there are many data items that can be operated on at the same time.
2. *Task-level parallelism (TLP)* arises because tasks of work are created that can operate independently and largely in parallel.

Computer hardware in turn can exploit these two kinds of application parallelism in four major ways:

1. *Instruction-level parallelism* exploits data-level parallelism at modest levels with compiler help using ideas like pipelining and at medium levels using ideas like speculative execution.
2. *Vector architectures, graphic processor units (GPUs), and multimedia instruction sets* exploit data-level parallelism by applying a single instruction to a collection of data in parallel.
3. *Thread-level parallelism* exploits either data-level parallelism or task-level parallelism in a tightly coupled hardware model that allows for interaction between parallel threads.
4. *Request-level parallelism* exploits parallelism among largely decoupled tasks specified by the programmer or the operating system.

Seven Great Ideas in Computer Architecture

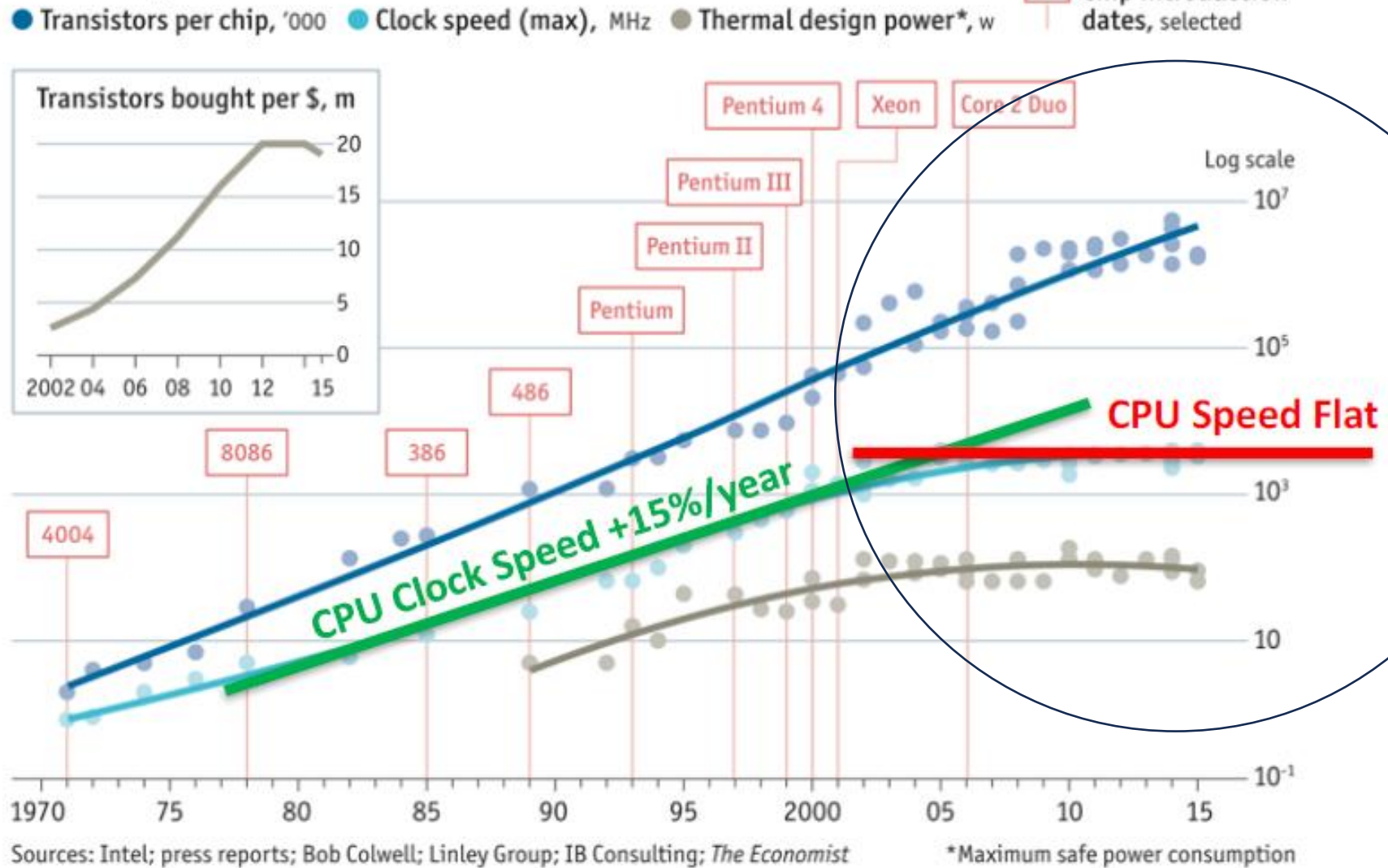
Explore further
In this course

1. Design for **Moore's Law**
2. Use **abstraction** to simplify design
3. Make the **common case fast**
4. Performance **via parallelism**
5. Performance **via pipelining**
6. Performance **via prediction**
7. **Hierarchy** of memories
8. **Dependability** via redundancy



Why is Computer Architecture Exciting Today?

Stuttering



Remaining Lecture

Interesting Topics

1. Discussion on Moore's Law, through paper 'MORE THAN MOORE' by M. Mitchell Waldrop, published In Nature, February 2016
2. Computer Performance Graphs from National Academy, USA, report
3. Introduction to the specialization area of 'Computer Architecture' through paper by Hennessy and Patterson, 'A New Golden Age for Computer Architecture', published in Communications of the ACM, February 2019, pages 48 to 60.

Important Directions for Computers of the future:

- a. Moore's Law failing to keep up
 - b. More Energy dissipation – too hot chips
 - c. Domain Specific Architectures
 - d. Enhanced security features inside microprocessors
 - e. Open Instruction Sets and Extension of Instruction Sets through Customized Accelerators
 - f. Agile Hardware Design for Microprocessors
 - g. Combination of Domain Specific Languages and Domain Specific Architectures
4. Intel Processor Timeline
 5. Reviewed course outline including detailed topics and grading breakup of lectures and labs.



Innovations like domain-specific hardware, enhanced security, open instruction sets, and agile chip development will lead the way.

BY JOHN L. HENNESSY AND DAVID A. PATTERSON

A New Golden Age for Computer Architecture

WE BEGAN OUR Turing Lecture June 4, 2018¹¹ with a review of computer architecture since the 1960s. In addition to that review, here, we highlight current challenges and identify future opportunities, projecting another golden age for the field of computer architecture in the next decade, much like the 1980s when we did the research that led to our award, delivering gains in cost, energy, and security, as well as performance.

"Those who cannot remember the past are condemned to repeat it." —George Santayana, 1901

Software talks to hardware through a vocabulary called an instruction set architecture (ISA). By the early 1960s, IBM had four incompatible lines of computers, each with its own ISA, software stack, I/O system, and market niche—targeting small business, large business, scientific, and real time, respectively. IBM

engineers, including ACM A.M. Turing Award laureate Fred Brooks, Jr., thought they could create a single ISA that would efficiently unify all four of these ISA bases.

5 They needed a technical solution
for how computers as inexpensive as

» key insights

- Software advances can inspire architecture innovation.
- Elevating the hardware/software interface creates opportunities for architecture innovation.
- The marketplace ultimately settles architecture debates.

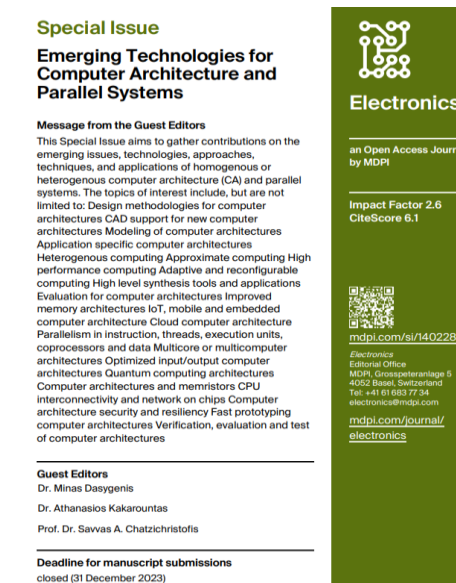
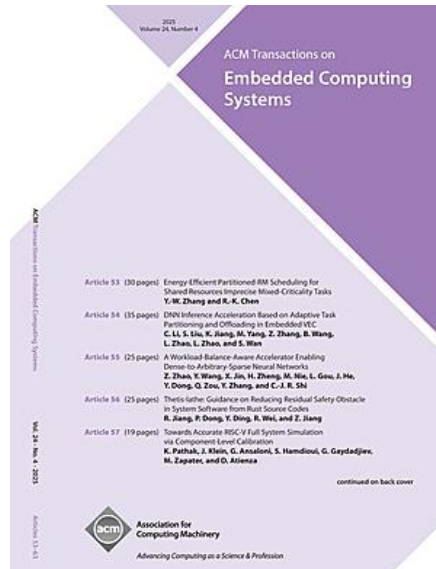
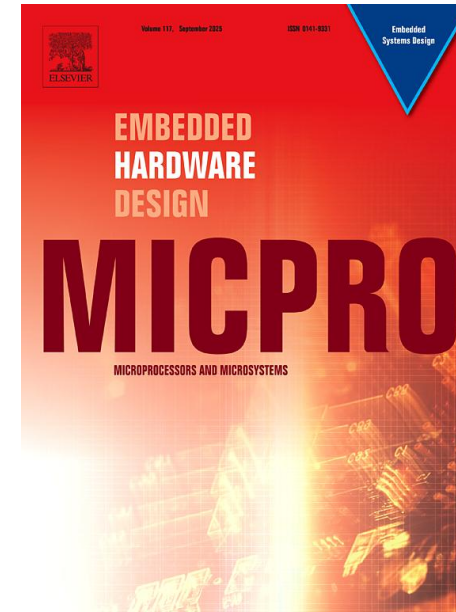
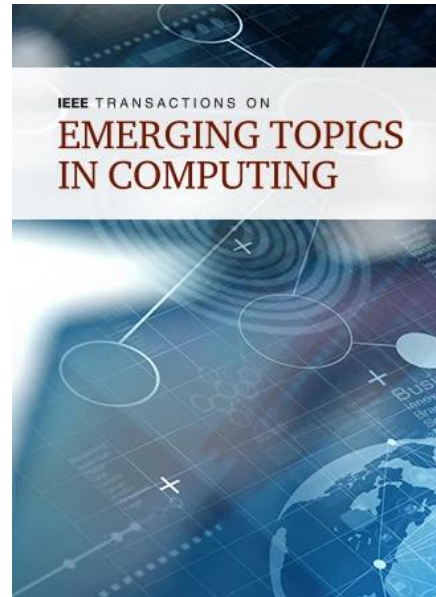
Video of Turing Lecture by Patterson, 2019

<https://dl.acm.org/doi/10.1145/3282307>

[David Patterson - A New Golden Age for Computer Architecture: History, Challenges and Opportunities – YouTube](#)



Some Computer Architecture Journals



IEEE Computer Society

ACM SIGARCH

Scale in Computing

The Scale In Computing



Decimal term	Abbreviation	Value	Binary term	Abbreviation	Value	% Larger
kilobyte	KB	10^3	kibibyte	KiB	2^{10}	2%
megabyte	MB	10^6	mebibyte	MiB	2^{20}	5%
gigabyte	GB	10^9	gibibyte	GiB	2^{30}	7%
terabyte	TB	10^{12}	tebibyte	TiB	2^{40}	10%
petabyte	PB	10^{15}	pebibyte	PiB	2^{50}	13%
exabyte	EB	10^{18}	exbibyte	EiB	2^{60}	15%
zettabyte	ZB	10^{21}	zebibyte	ZiB	2^{70}	18%
yottabyte	YB	10^{24}	yobibyte	YiB	2^{80}	21%

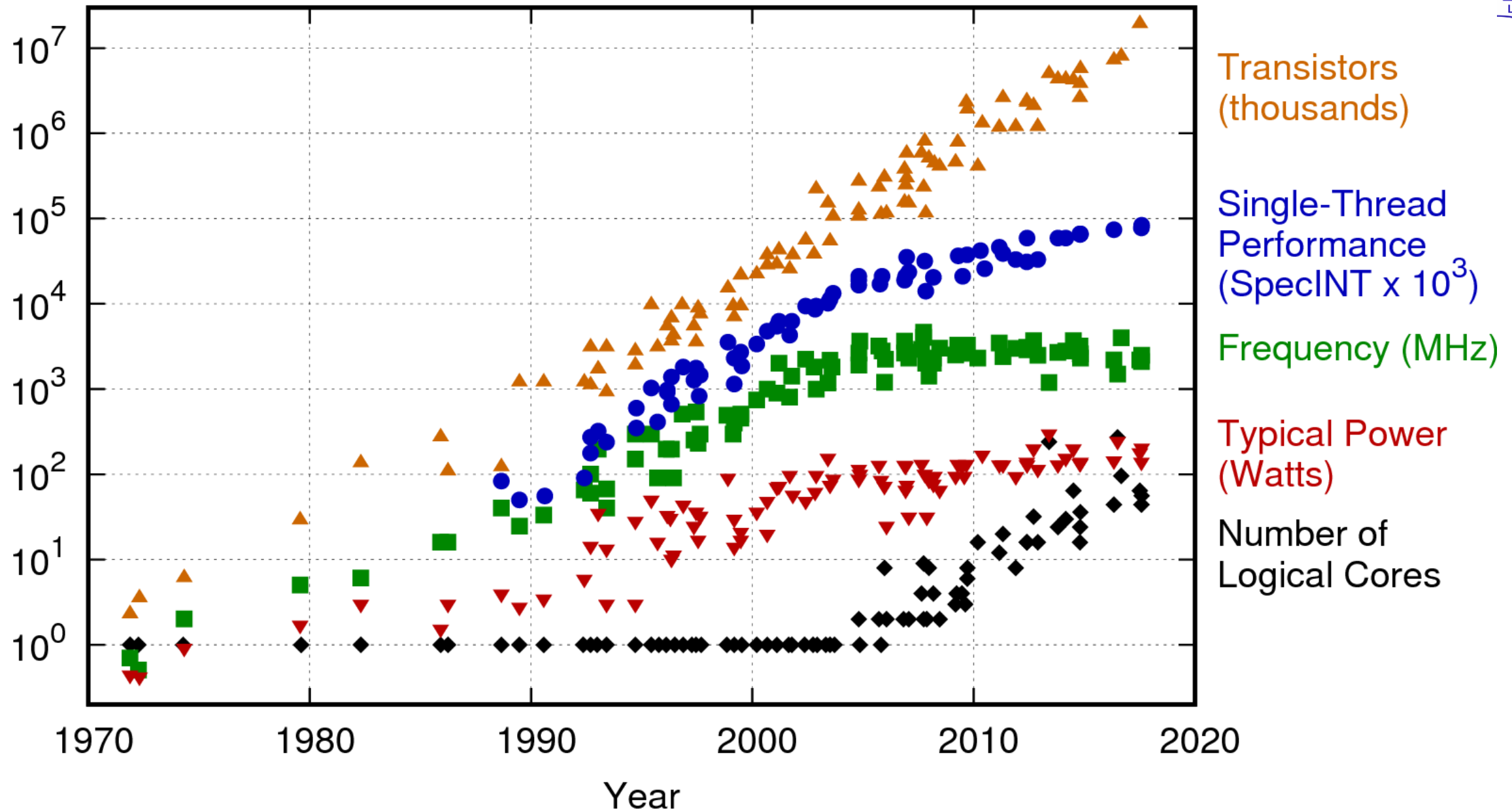
FIGURE 1.1 The 2^x vs. 10^y bytes ambiguity was resolved by adding a binary notation for all the common size terms. In the last column we note how much larger the binary term is than its corresponding decimal term, which is compounded as we head down the chart. These prefixes work for bits as well as bytes, so *gigabit* (Gb) is 10^9 bits while *gibibits* (Gib) is 2^{30} bits.

Approximations in Huge Binary Numbers



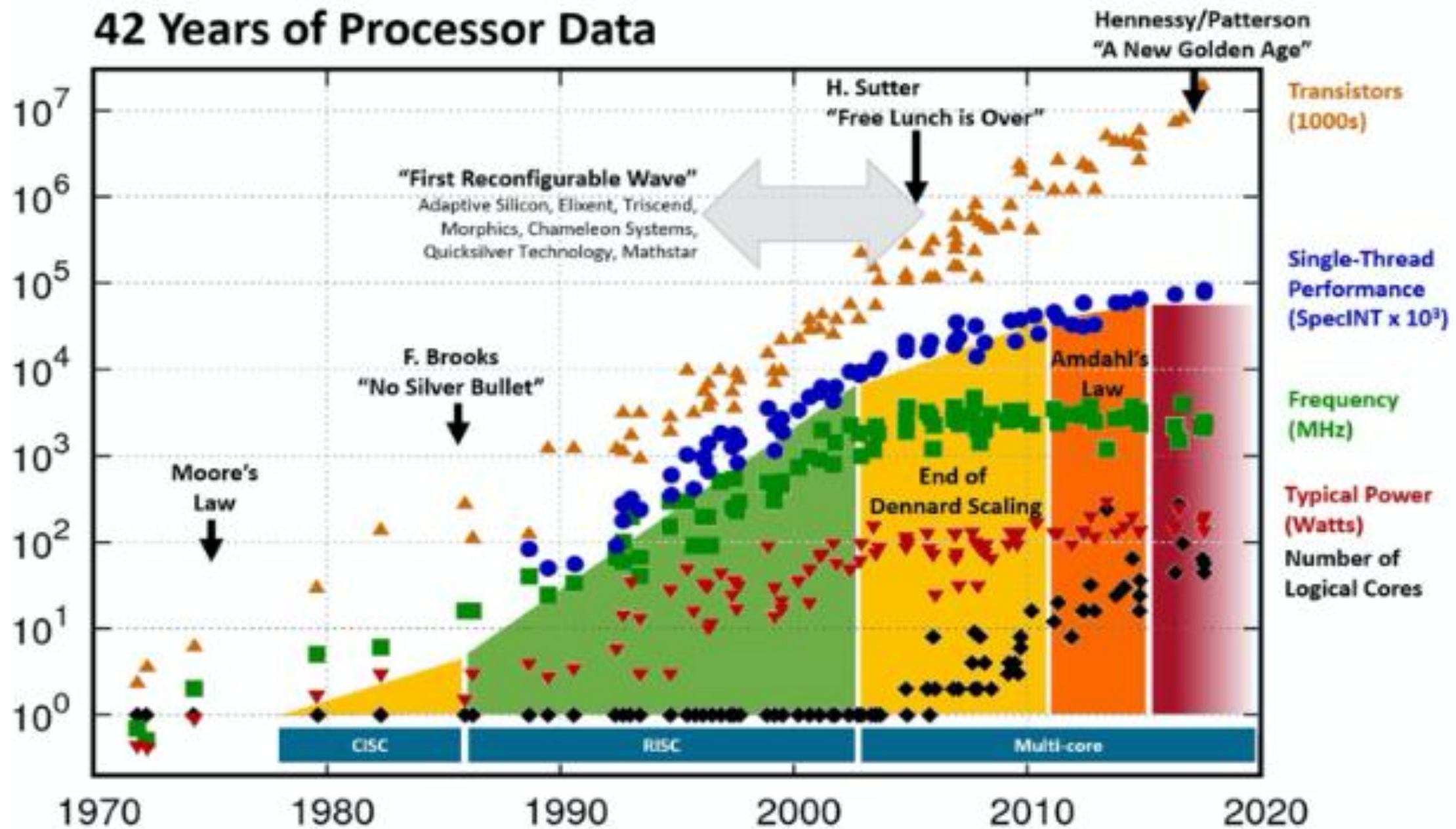
- 2^{10} is approximately 1 Kilo (approx 1×10^3)
- 2^{20} is approximately 1 Mega (approx 1×10^6)
- 2^{30} is approximately 1 Giga (approx 1×10^9)

42 Years of Microprocessor Trend Data



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2017 by K. Rupp

42 Years of Processor Data

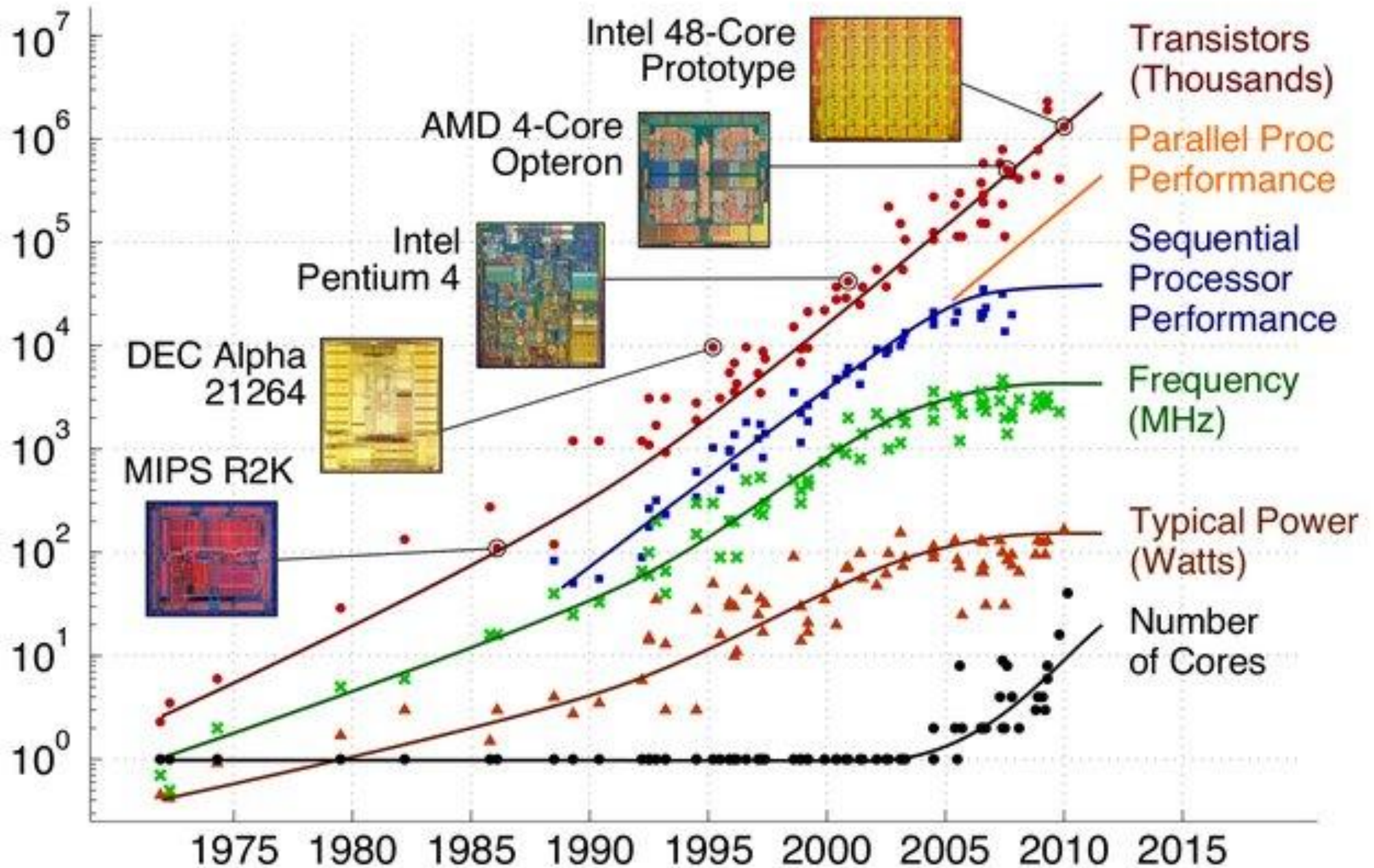


Hennessy and Patterson, Turing Lecture 2018, overlaid over "42 Years of Processors Data"

<https://www.karlrupp.net/2018/02/42-years-of-microprocessor-trend-data/>; "First Wave" added by Les Wilson, Frank Schirrmeister

Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten

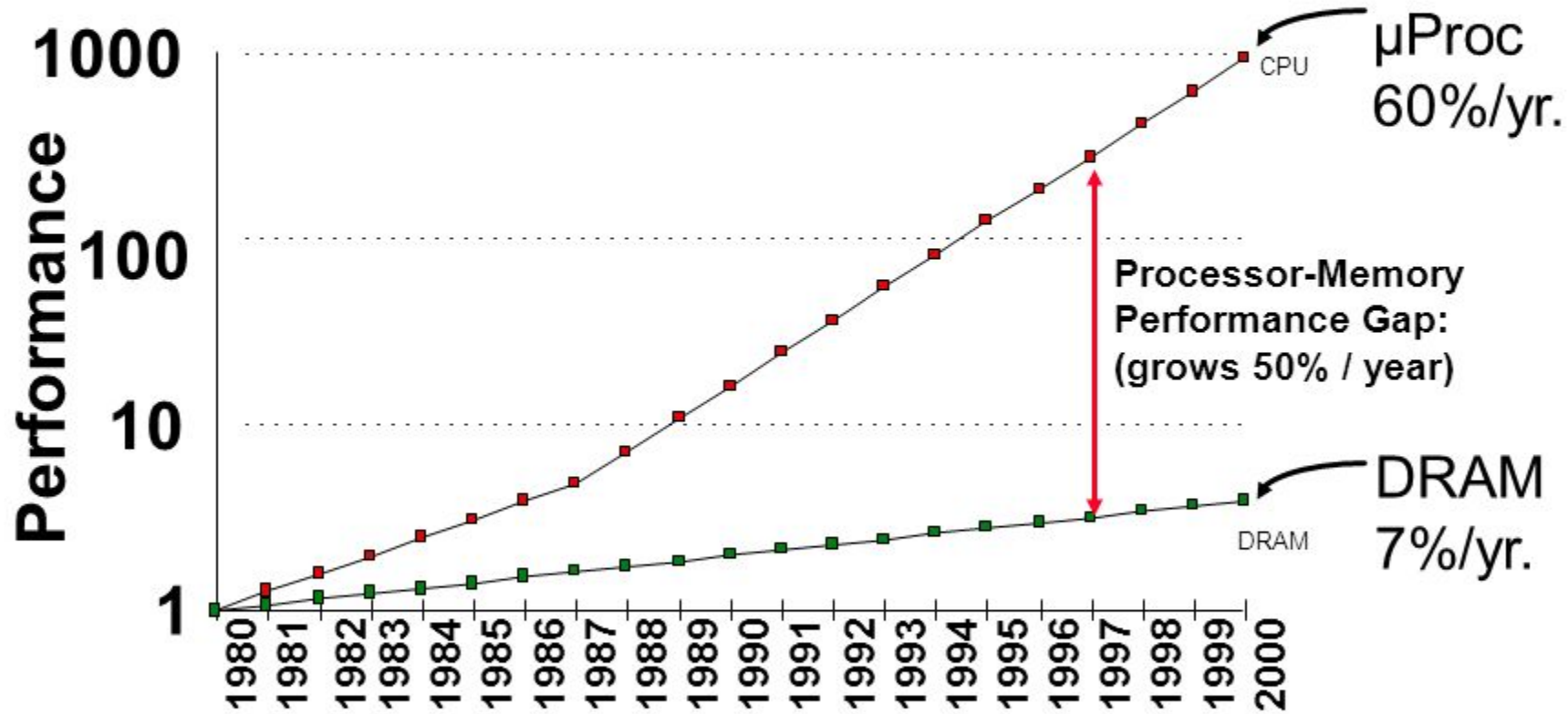
New plot and data collected for 2010-2017 by K. Fournier



Memory Hierarchy: Motivation

Processor-Memory (DRAM) Performance Gap

(i.e. Gap between memory access time and CPU cycle time)



EECC551 - Shaaban

Introducing Instruction Set Architecture (ISA)

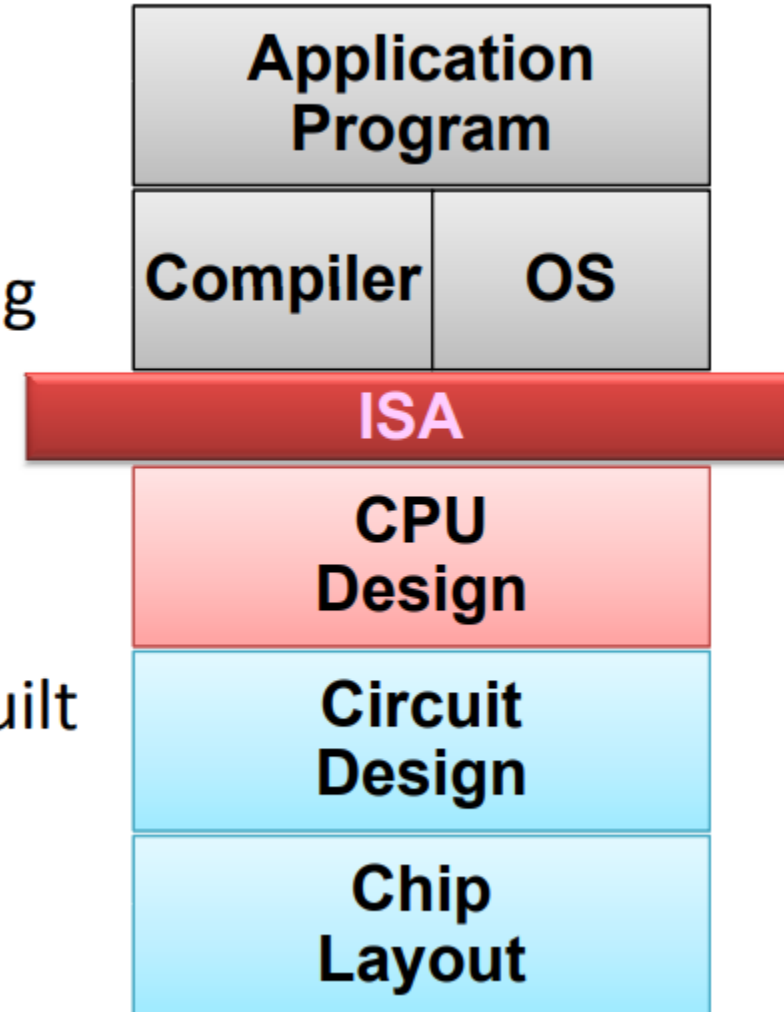
What is an Instruction Set Architecture?



- Lowest level of Computer Architecture that is visible to a programmer
- ISA comprises instructions in Assembly Language and Corresponding Binary Machine Code
- Primitive syntax compared to a high-level language
- It is easily interpreted and understood by the CPU
- Designed to maximize performance
- Represents features and performance of a CPU
- Designed to minimize cost and design time

Role of ISA within a Computer

- Assembly Language View
 - Processor state (RF, mem)
 - Instruction set and encoding
- Layer of Abstraction
 - Above: how to program machine - HLL, OS
 - Below: what needs to be built - tricks to make it run fast



Key:

RF = Register File

Mem = Memory

HLL = High Level Language

OS = Operating System

Instruction Set Architecture: The Myopic View of Computer Architecture

We use the term *instruction set architecture* (ISA) to refer to the actual programmer-visible instruction set in this book. The ISA serves as the boundary between the software and hardware. This quick review of ISA will use examples from 80x86, ARMv8, and RISC-V to illustrate the seven dimensions of an ISA. The most popular RISC processors come from ARM (Advanced RISC Machine), which were in 14.8 billion chips shipped in 2015, or roughly 50 times as many chips that shipped with 80x86 processors. Appendices A and K give more details on the three ISAs.

RISC-V (“RISC Five”) is a modern RISC instruction set developed at the University of California, Berkeley, which was made free and openly adoptable in response to requests from industry. In addition to a full software stack (compilers, operating systems, and simulators), there are several RISC-V implementations freely available for use in custom chips or in field-programmable gate arrays. Developed 30 years after the first RISC instruction sets, RISC-V inherits its ancestors’ good ideas—a large set of registers, easy-to-pipeline instructions, and a lean set of operations—while avoiding their omissions or mistakes. It is a free and open, elegant example of the RISC architectures mentioned earlier, which is why more than 60 companies have joined the RISC-V foundation, including AMD, Google, HP Enterprise, IBM, Microsoft, Nvidia, Qualcomm, Samsung, and Western Digital. We use the integer core ISA of RISC-V as the example ISA in this book.

- Data Alignment
 - 32-bit bus vs 8-bit memory
 - Big Endian vs Little Endian
- Addressing Modes
- Type and Size of Operands
 - I, R types
- Operations available
 - Arithmetic
 - Logical
 - Shift
- Control Flow Instructions
 - Branch
 - Jump

Memory Access – Big Endian vs Little Endian

A CPU Register **R0** is 32-bits wide.

It has data “1A2B3C4D” Hex.

Store the data in an 8-bit wide RAM that has 4 locations.

Store in **Big-Endian style**

or Store in Little-Endian Style

CPU Register
R0 Contents



Location	Big Endian RAM
3	
2	
1	
0	

Location	Little Endian RAM
3	
2	
1	
0	

RISC-V Registers and Naming Conventions



Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-x11	a0-a1	Function arguments/return values	Caller
x12-x17	a2-a7	Function arguments	Caller
x18-x27	s2-s11	Saved registers	Callee
x28-x31	t3-t6	Temporaries	Caller
f0-f7	ft0-ft7	FP temporaries	Caller
f8-f9	fs0-fs1	FP saved registers	Callee
f10-f11	fa0-fa1	FP function arguments/return values	Caller
f12-f17	fa2-fa7	FP function arguments	Caller
f18-f27	fs2-fs11	FP saved registers	Callee
f28-f31	ft8-ft11	FP temporaries	Caller

RISC-V is a Register-rich Architecture

Figure 1.4 RISC-V registers, names, usage, and calling conventions. In addition to the 32 general-purpose registers (x0–x31), RISC-V has 32 floating-point registers (f0–f31) that can hold either a 32-bit single-precision number or a 64-bit double-precision number. The registers that are preserved across a procedure call are labeled “Callee” saved.

RISC-V Instruction Formats

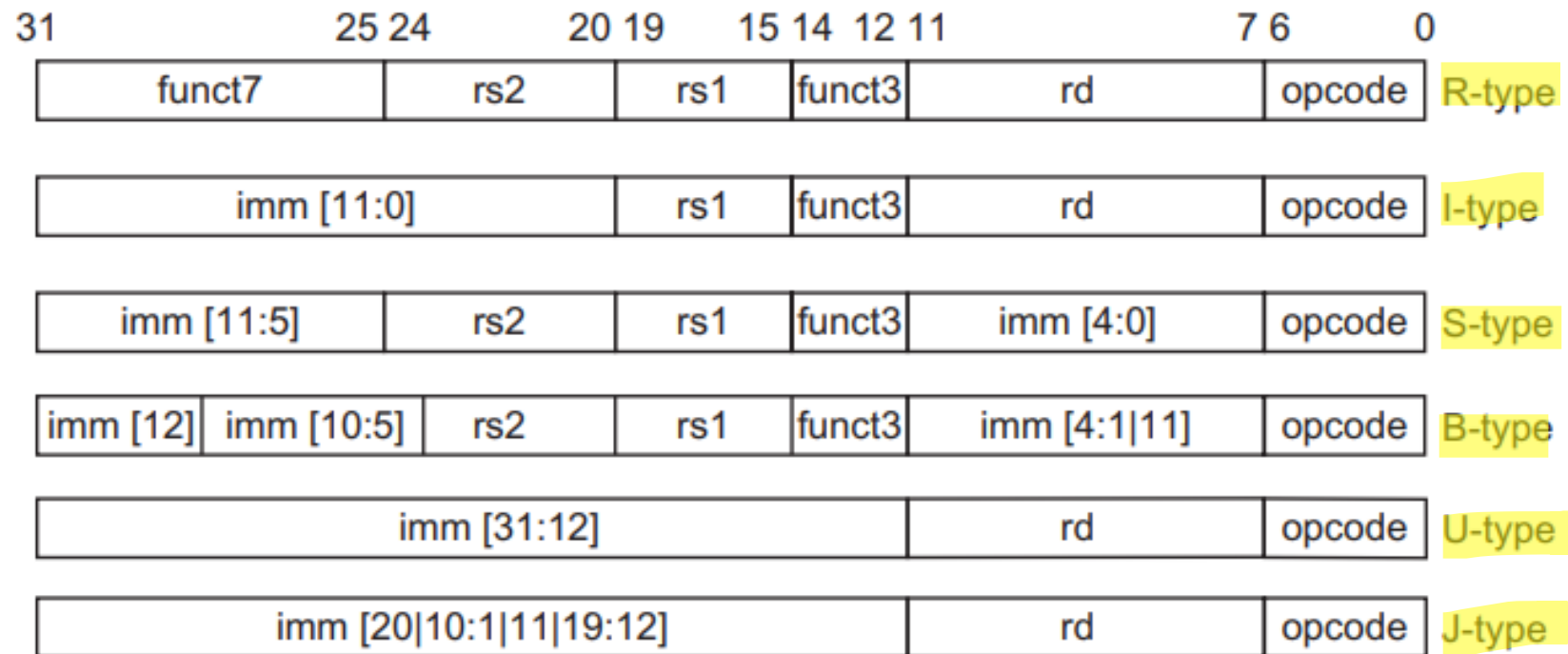


Figure 1.7 The base RISC-V instruction set architecture formats. All instructions are 32 bits long. The R format is for integer register-to-register operations, such as ADD, SUB, and so on. The I format is for loads and immediate operations, such as LD and ADDI. The B format is for branches and the J format is for jumps and link. The S format is for stores. Having a separate format for stores allows the three register specifiers (rd, rs1, rs2) to always be in the same location in all formats. The U format is for the wide immediate instructions (LUI, AUIPC).

Instruction type/opcode	Instruction meaning
<i>Data transfers</i>	<i>Move data between registers and memory, or between the integer and FP or special registers; only memory address mode is 12-bit displacement + contents of a GPR</i>
lb, lbu, sb	Load byte, load byte unsigned, store byte (to/from integer registers)
lh, lhu, sh	Load half word, load half word unsigned, store half word (to/from integer registers)
lw, lwu, sw	Load word, load word unsigned, store word (to/from integer registers)
ld, sd	Load double word, store double word (to/from integer registers)
flw, fld, fsw, fsd	Load SP float, load DP float, store SP float, store DP float
fmv.__.x, fmv.x. __	Copy from/to integer register to/from floating-point register; “__” = S for single-precision, D for double-precision
csrrw, csrrwi, csrrs, csrrsi, csrrc, csrrci	Read counters and write status registers, which include counters: clock cycles, time, instructions retired

RISC-V Arithmetic / Logical Instructions



Arithmetic/logical

add, addi, addw, addiw

sub, subw

mul, mulw, mulh, mulhsu,
mulhu

div, divu, rem, remu

divw, divuw, remw, remuw

and, andi

or, ori, xor, xori

lui

auipc

sll, slli, srl, srli, sra,
srai

sllw, slliw, srlw, srliw,
sraw, sraiw

slt, slti, sltu, sltiu

Operations on integer or logical data in GPRs

Add, add immediate (all immediates are 12 bits), add 32-bits only & sign-extend to 64 bits, add immediate 32-bits only

Subtract, subtract 32-bits only

Multiply, multiply 32-bits only, multiply upper half, multiply upper half signed-unsigned, multiply upper half unsigned

Divide, divide unsigned, remainder, remainder unsigned

Divide and remainder: as previously, but divide only lower 32-bits, producing 32-bit sign-extended result

And, and immediate

Or, or immediate, exclusive or, exclusive or immediate

Load upper immediate; loads bits 31-12 of register with immediate, then sign-extends

Adds immediate in bits 31-12 with zeros in lower bits to PC; used with JALR to transfer control to any 32-bit address

Shifts: shift left logical, right logical, right arithmetic; both variable and immediate forms

Shifts: as previously, but shift lower 32-bits, producing 32-bit sign-extended result

Set less than, set less than immediate, signed and unsigned

Control

beq, bne, blt, bge, bltu,
bgeu

Conditional branches and jumps; PC-relative or through register

Branch GPR equal/not equal; less than; greater than or equal, signed and unsigned

jal, jalr

Jump and link: **save PC+4**, **target is PC-relative** (JAL) or a register (JALR); if specify x0 as destination register, then acts as a simple jump

ecall

Make a request to the supporting execution environment, which is usually an OS

ebreak

Debuggers used to cause control to be transferred back to a debugging environment

fence, fence.i

Synchronize threads to guarantee ordering of memory accesses; synchronize instructions and data for stores to instruction memory

Performance of Computer Systems

- Time to execute a program – in seconds
- Clock Speed of Computer – Mega Hertz or Giga Hz
- Comparing Benchmarks
- In terms of the width of Data Bus (16 bit, 32 bit, 64 bit)
- In terms of the size of different memory (8GB / 256GB)
 - Cache size
 - RAM
 - Hard disk
 - SSD, etc.
- In terms of multiple cores and multiple I/O available
- Software
 - Algorithm
 - Language / Compiler
 - Optimization – Parallel, Vector, etc.

**In terms of
Resources**

What is MFLOPS or Giga Flops or Tera Flops?



FLOPS = Floating Point Linear Operations Per Second

$$\text{MFLOPS rate} = \frac{\text{Number of executed floating point operations in a program}}{\text{Execution time} \times 10^6}$$

- Take Advantage of Parallelism
 - e.g. multiple processors, disks, memory banks, pipelining, multiple functional units
- Principle of Locality
 - Reuse of data and instructions (i.e. in Caches)
- Focus on the Common Case
 - Amdahl's Law

$$Speedup_{overall} = \frac{1}{(1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}}$$

Energy and Power

CPU Architecture Today

Heat becoming an unmanageable problem



Power Wall

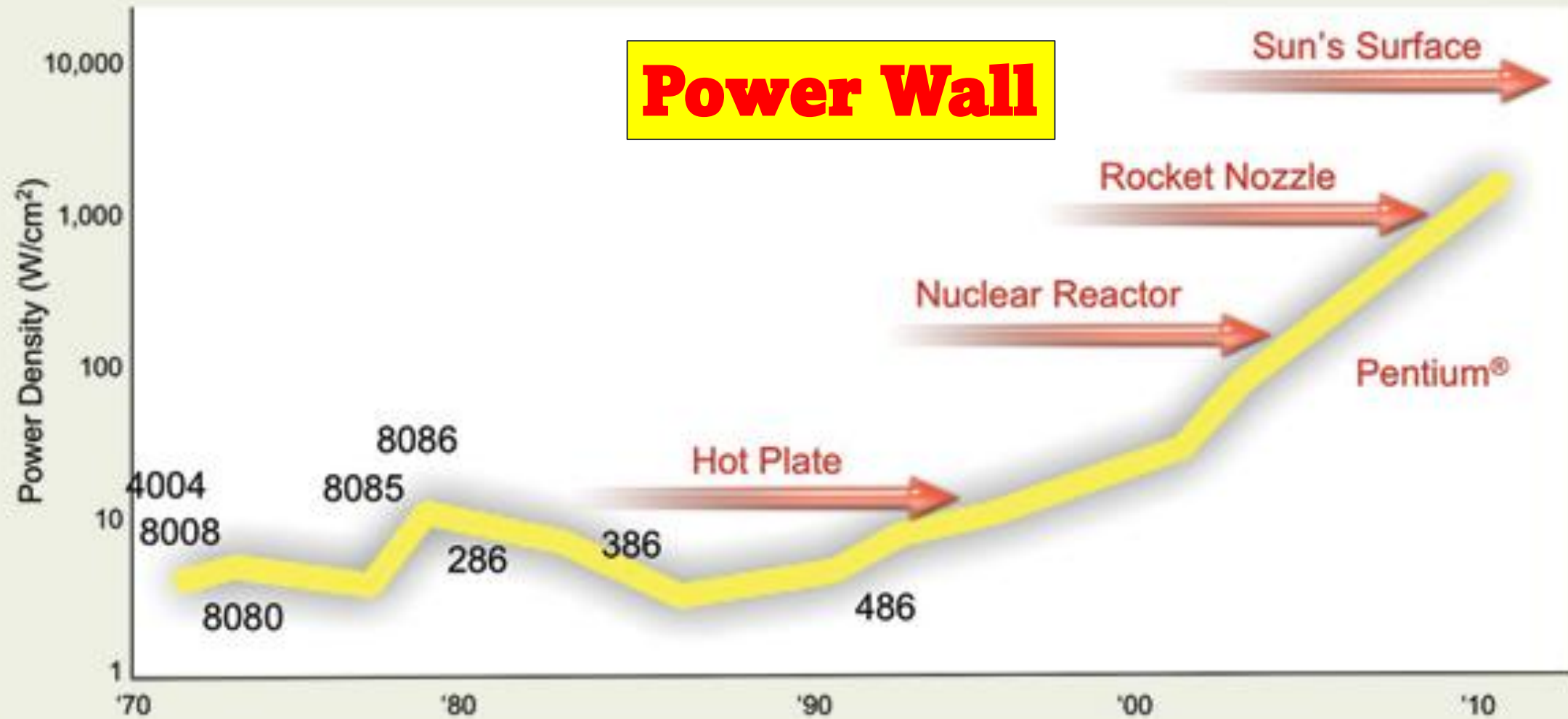


Figure 1. In CPU architecture today, heat is becoming an unmanageable problem.
(Courtesy of Pat Gelsinger, Intel Developer Forum, Spring 2004)

Energy and Power Within a Microprocessor

For CMOS chips, the traditional primary energy consumption has been in switching transistors, also called *dynamic energy*. The energy required per transistor is proportional to the product of the capacitive load driven by the transistor and the square of the voltage:

$$\text{Energy}_{\text{dynamic}} \propto \text{Capacitive load} \times \text{Voltage}^2$$

This equation is the energy of pulse of the logic transition of $0 \rightarrow 1 \rightarrow 0$ or $1 \rightarrow 0 \rightarrow 1$. The energy of a single transition ($0 \rightarrow 1$ or $1 \rightarrow 0$) is then:

$$\text{Energy}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2$$

The power required per transistor is just the product of the energy of a transition multiplied by the frequency of transitions:

$$\text{Power}_{\text{dynamic}} \propto 1/2 \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

For a fixed task, slowing clock rate reduces power, but not energy.

Clearly, dynamic power and energy are greatly reduced by lowering the voltage, so voltages have dropped from 5 V to just under 1 V in 20 years. The capacitive load is a function of the number of transistors connected to an output and the technology, which determines the capacitance of the wires and the transistors.

Example of Energy calculation



Example Some microprocessors today are designed to have adjustable voltage, so a 15% reduction in voltage may result in a 15% reduction in frequency. What would be the impact on dynamic energy and on dynamic power?

Answer Because the capacitance is unchanged, the answer for energy is the ratio of the voltages

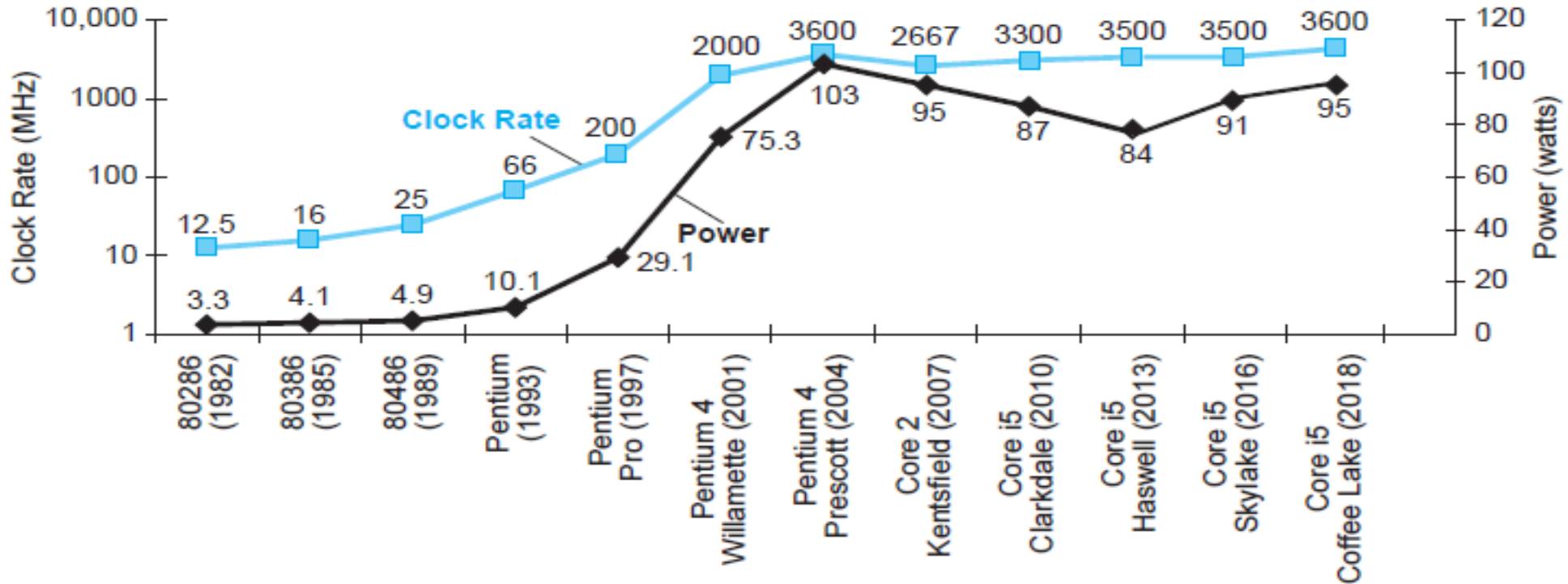
$$\frac{\text{Energy}_{\text{new}}}{\text{Energy}_{\text{old}}} = \frac{(\text{Voltage} \times 0.85)^2}{\text{Voltage}^2} = 0.85^2 = 0.72$$

which reduces energy to about 72% of the original. For power, we add the ratio of the frequencies

$$\frac{\text{Power}_{\text{new}}}{\text{Power}_{\text{old}}} = 0.72 \times \frac{(\text{Frequency switched} \times 0.85)}{\text{Frequency switched}} = 0.61$$

shrinking power to about 61% of the original.

CPU Power Trends



- In CMOS IC technology

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

×30

5V → 1V

×1000

- H&P Computer Architecture Textbook chapter 1
- Research Papers for new Computing architectures and paradigm
 - Challenging Trends in Energy of Computing for Data Centers, IEEE Computer, 2024
 - The 50 Year History of the Microprocessor as Five Technology Eras, IEEE Micro, 2021
 - A New Golden Age for Computer Architecture, Turing Lecture, Communications of the ACM, 2019
- See end of chapter questions from H&P textbook